Autonomous and Adaptive Systems Project

Enrico Tornabene
Computer Engineering
Alma Mater Studiorum, University of Bologna
enrico.tornabene@studio.unibo.it

Abstract

This project studies the performance of four different DQN variants, A2C and MAPPO reinforcement learning algorithms across three different multi-agent social-dilemma environments: Stag Hunt, Harvest and Escalation. In the environment where successful coordination is rare like Stag Hunt, off-policy DQN variants have the highest performance, while on-policy methods struggle due to the rare-event discovery problem. In an environment with dense reward as Harvest, MAPPO and A2C achieve the highest total rewards but fail to learn to wait for plants to grow in order to maximize returns despite severe reward shaping. In the Escalation environment the coordination capabilities of MAPPO's Centralized Training Decentralized Execution (CTDE) paradigm are the key factor behind its superior performance.

1 Introduction

Stag-Hunt is a classical game that represents a *social dilemma cooperation*, presenting a conflict between a low-reward foraging option that is risk-free, and a high-reward hunting option that can be heavily penalizing. The only way to hunt the stag, and get the highest reward, is to cooperate with the other player, otherwise mauling occours and the player will be penalized.

The Harvest game is based on a resource management dynamics where agents must harvest plants that grow over time. Since young plants have a smaller reward than mature plants, agents must learn to delay immediate rewards and avoid collecting resources prematurely to maximize the cumulative reward.

The Escalation environment introduces a coordination challenge with high penalties for individual agent failure. Agents must coordinate their steps on their way to the stag. If a single agent fail a penalty is applied.

1.1 Background

For this project different families of algorithms are used: value-function approximation such as DQNs, Actor-Critic method and Proximal Policy Optimization method.

Deep Q-Learning (DQN) is an off-policy action-value approximation algorithm, which uses a neural network (NN) to estimate $\hat{q}(S_t, A_t, w)$ where w represents the weights of the network. The update is:

$$w_{t+1} = w_t + \eta[R_{t+1} + \gamma \max_{a \in A(S_{t+1})} \hat{q}(S_{t+1}, a, w_t) - \hat{q}(S_t, A_t, w_t)]\nabla_w \hat{q}(S_t, A_t, w_t) \quad (1)$$

where η is the step-size and γ is the discount factor, in practice this update is done by minimizing the *Mean Squarred Error (MSE)* between the TD-target and the current Q-values estimation via backpropagation. It uses a *Replay Memory* to overcome the problem of high correlation between subsequent samples. The *vanilla* formulation of DQN suffers from the *moving target problem*: it uses the same network parameters to estimate the current Q-values and the target. To solve this problem,

Table 1: Hyperparameters used in the experiments.

Learning Rate (α)	Discount Factor (γ)	Replay Buffer	Batch Size	Target Net. Update (C)	PPO Clip (ϵ)	GAE Param. (λ)	Entropy Coeff.
10^{-3}	0.99	10^4	64	500	0.2	0.95	0.01

the **Standard DQN** implementation decouples the Q-values estimation (policy network) from the target generation, using a target network which is updated only every C steps. **Double DQN** reduces the overestimation of Q-values due to the max operator, decoupling the action selection, done by the policy network, and the action evaluation, done by the target network. Lastly, the **Dueling DQN** architecture separates the estimation of the state-value $V(s)$ and the estimation of the advantage function $A(s, a)$ (how good an action is with respect to the average) to improve further the learning and the generalization. This is achieved having two different heads in the NN architecture, after a common feature extraction part.

Advantage Actor Critic (A2C) is an on-policy method which learns to approximate both the policy (Actor) and the value function (Critic). The value function is used as a baseline in order to reduce variance. A2C relies on the *advantage function* to evaluate how much better the selection of action A_t in state S_t is compared to the average value of S_t:

$$\delta_{ADV} = \hat{Q}(S_t, A_t, w) - \hat{v}(S_t, w) \tag{2}$$

Since the true action-value $Q(S_t, A_t)$ is unknown, it is estimated using the discounted return G_t. The optimization is done by combining the actor loss, the critic loss and an *entropy* term to encourage exploration.

Multi Agent Proximal Policy Optimization (MAPPO) is an on-policy algorithm which uses first-order optimization method to keep new policies close to the old ones, by clipping the actor loss function:

$$L^{CLIP} = \min(\frac{\pi_\theta(a|s)}{\pi_{\theta_t}(a|s)}\delta^{\pi_{\theta_t}}_{ADV}(s,a), clip(\frac{\pi_\theta(a|s)}{\pi_{\theta_t}(a|s)}, 1-\epsilon, 1+\epsilon)\delta^{\pi_{\theta_t}}_{ADV}(s,a)) \tag{3}$$

where ϵ is the clipping value. MAPPO is based on the *Centralized Training Decentralized Execution (CTDE)* paradigm: each actor has access only to its local observation (decentralized execution), while a centralized critic observes the global state during training (joint state). The implementation uses the **Generalized Advantage Estimation (GAE)** as advantage function:

$$\hat{\delta}^{GAE}_t = \sum_{l=0}^{T} (\gamma\lambda)^l \delta_{t+l} \tag{4}$$

where δ_t is the TD-error at time t, γ is the discount factor and λ is the GAE parameter.

Both A2C and MAPPO share the same problem: as on-policy methods, they evaluate the returns of their current policy and do not have the optimistic bias of the off-policy algorithms ($\max$ operator over future Q-values). Their learning depends on the frequency of highly rewarded events that occour during rollouts. If the optimal (joint) action requires coordination between agents, and has a very low probability of being taken, these algorithms often fail to discover the optimal strategy, converging to safer equilibrium (foraging).

2 Stag-Hunt experiments and analisys

For the stag-hunt environment various test are done, changing the grid-size and the rewards values, while the forage quantity is kept constant at 2. The algorithms with ϵ-greedy have a decreasing behaviour, the exploration rate ϵ decays from 1.0 to 0.01. The algorithms are evaluated using the hyperparameters summarized in Table 1.

For the experiments a constant seed is used to ensure reproducibility (`seed=42`).

2.1 Different Algorithms Results on Stag-Hunt

As shown in figure 1, in a Stag-Hunt environment with a grid-size of 5×5, a reward value for stag hunting, for foraging and a mauling penalty equal respectively to $5, 1, -3$, Double DQN and Dueling

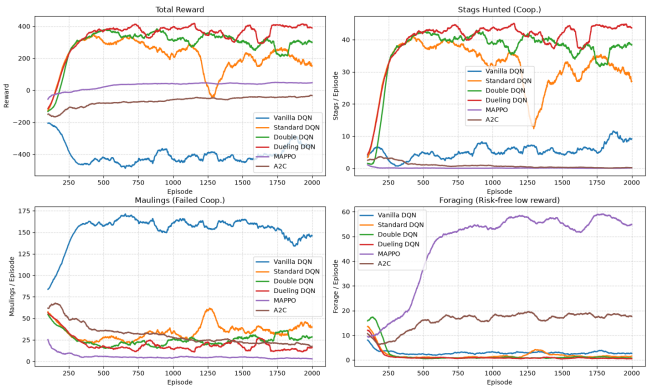
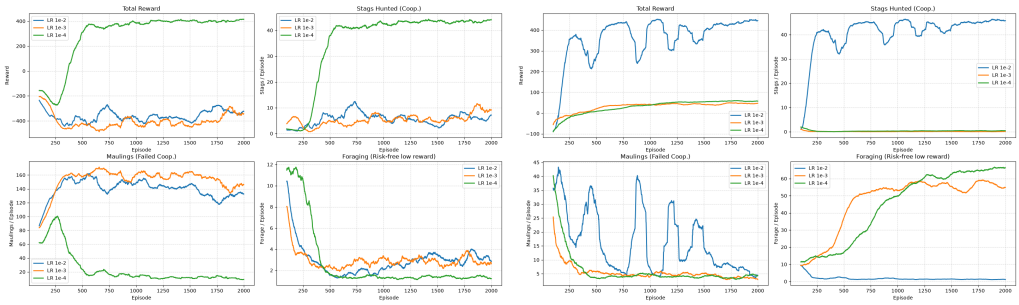


Figure 1: Training results for every algorithm on standard environment configuration using hypeparameters listed in Table 1 in Stag Hunt game.



(a) Vanilla DQN (b) MAPPO

Figure 2: Comparison of learning rate effects between Vanilla DQN and MAPPO in Stag Hunt game.

DQN converge fast to the optimal solution reaching the highest reward thanks to the high number of stags hunted. Standard DQN algorithm has a similar trajectory but has a collapse leading it to a lower total reward from which it partially recovers. The Vanilla DQN totally fails, its reward drops instantly due to the high and constant number of maulings. This behaviour is a consequence of the moving target problem.

Neither on-policy approach discovers the stag hunting under this configuration. MAPPO converges to a pure foraging solution, while A2C converges slowly to a mixed solution of maulings and foraging.

2.2 Learning Rate effects

Here are presented the results of studying different values of learning rates $\alpha \in \{10^{-2}, 10^{-3}, 10^{-4}\}$ for Vanilla DQN and for MAPPO. The resultst are shown in Figure 2.

Vanilla DQN increases the number of stags hunted as long as the learning rate is small due to the moving target problem: if the vanilla algorithm uses large learning rates the updates at every step are drastic and so the target heavily changes; if it uses small learning rate values the update are smaller, allowing the algorithm to converge more reliably (though more slowly) instead of diverging.

MAPPO achieves a higher number of hunted stags using higher learning rates value. An accidental initial stag hunt produces a huge gradient update that significantly changes the policy before the agents get stuck in the foraging equilibrium. The main drawback of this approach is the high instability shown in the in Figure 2b.

2.3 Reward Sensitivity

Figure 3 shows only the results for changing the reward and punishment value on MAPPO but experiments are conducted also on Dueling DQN and A2C.

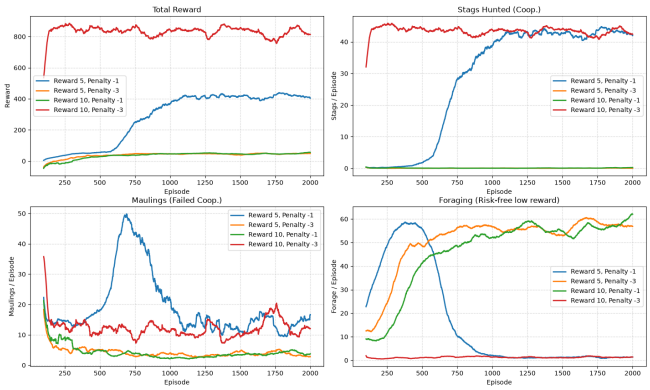
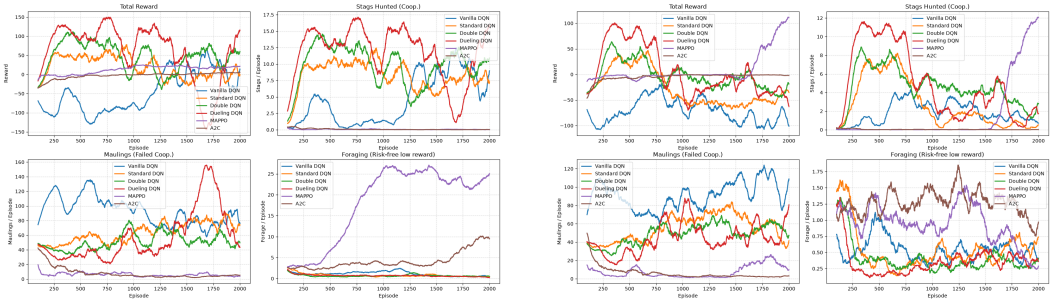


Figure 3: Convergence analysis of MAPPO with varying rewards and punishments in Stag Hunt game.



(a) 10×10 Grid

(b) 15×15 Grid

Figure 4: Comparison of performance with different grid sizes in Stag Hunt game.

Dueling DQN discovers the optimal stag equilibrium in all four configurations, converging rapidly ($\sim$ 250 episodes) regardless of the value of the reward or penalty. This is due to the max-based bootstrap and the experience replay buffer discussed earlier, which allows rare coordination steps to be re-sampled and reinforced.

MAPPO presents a different behaviour: cooperation emerges with configuration $(5, -1)$ and $(10, -3)$ but converges to a safe foraging strategy with both $(5, -3)$ and $(10, -1)$. It is not fully explainable why the most permissive configuration $(10, -1)$ fails: it may happen because the stag reward is too high compared to the penalty.

A2C exhibits a similar behaviour to MAPPO succeeding with the same two configurations, $(5, -1)$ and $(10, -3)$, and failing under $(10, -1)$. The main difference is with the $(5, -3)$ configuration: differently from MAPPO, A2C discovers cooperation ($\sim$ 1500 episodes). Like MAPPO, A2C fails on $(10, -1)$, seeing this same failure on two different algorithms may suggest that $(10, -1)$ is an anomaly.

The tests are conducted with a single seed (42). Future work should evaluate the behavior with $(10, -1)$ configuration across multiple seeds.

2.4 Grid Size scaling

To test the effect of different grid sizes, the reward setup $(5, -1)$ is used, in order to start with a configuration which at smallest grid size successfully converge to stag optimum even for the on-policy algorithms. In Figure 4 are reported only the results for (10×10) and (15×15) sizes.

The DQNs family decrease their performance as the grid size increases. Dueling DQN's stag-hunting rate decreases but remain above zero. On the other hand, both on-policy methods collapse almost immediately with bigger grid-sizes. At (15×15), MAPPO shows a late cooperation behaviour at the

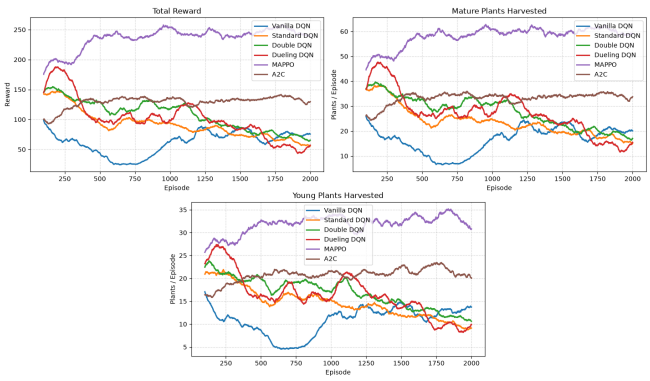


Figure 5: Performance comparison of all six algorithm in Harvest environment.

end of the training ($\sim$ 1700 episodes). Larger grids reduce the probability of random joint encounters with stags. A2C instead fails to establish any strategy with any configuration.

3 Results on different environments

3.1 Harvest

For the Harvest game the algorithms are tested on a (5×5) grid size, with mature plant and young plant harvested reward respectively of 3 and 1. Results are shown in Figure 5.

Differently from Stag-Hunt game, where most of DQN variants outperformed the on-policy methods with the default reward configuration, MAPPO and A2C dominate the training on Harvest, ranking respectively first and second, with respect to the total reward gained. This happens because at every step numerous harvesting events happen, being more common than the stag hunting event, meaning that the algorithms do not suffer from rare event discovery.

A critical problem persisted with different configurations which were not able to induce delayed harvesting to maximize the reward. On-policy methods achieved the highest reward, failing to wait for plants to mature, falling into a *greedy* dynamic constantly harvesting younger plants. To solve this problem, different young plant harvesting reward shaping are tried, also applying high penalties (e.g., -5): on-policy methods "absorbed" the negative reward and still failed to learn to wait for plants to grow; on the other hand DQNs were completely broken by this reward shaping, learning to avoid harvesting.

3.2 Escalation

For the Escalation game the tests are conducted on a (5×5) grid size, with punishment factor of 0.5 which is multiplied for the total streak lenght T if one agent break the coordinations with the other one.

As shown in Figure 6 MAPPO outperforms all other algorithms reaching streak of $\sim$ 60 steps. Among DQNs variants, the Dueling DQN is the most stable being the only algorthm that find cooperation and mantains it during all the training time. Double DQN finds cooperation but a sharp performance decrease occurs mid training, Standard DQN follows a similar trajectory before decreasing to zero cooperation ($\sim$ 750 episode). Vanilla DQN fails to cooperate, consistent with the instability caused by the lack of a target network. A2C totally fails to achieve cooperation throughout training, a constant results with different punishment factor values (e.g., 0.1).These results suggest that on-policy learning per se is not the main factor behind the difference observed throughout this work: MAPPO has a CTDE paradigm that can coordinate better agents in cooperation.

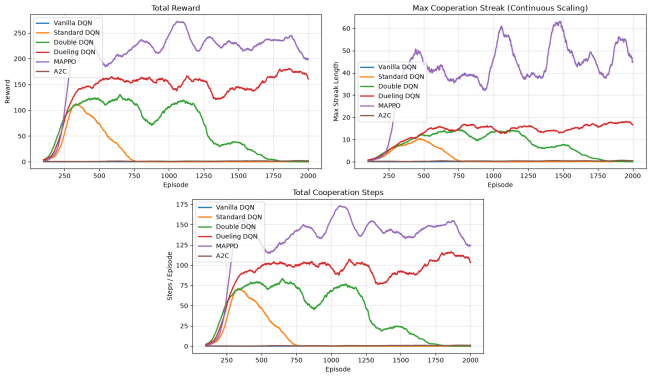


Figure 6: Performance comparison of all six algorithm in Escalation environment.

4 Conclusions

This project shows the trade-off in multi-agent reinforcement learning: DQN variants, particularly Dueling DQN, excel in sparse-reward environments like Stag-Hunt. This family of algorithms partially collapses in environments with strict synchronization requirements as in Escalation, where Standard and Double DQN fail to sustain cooperation while Dueling DQN remains robust . On the other hand, on-policy algorithms achieve high performance in dense-reward settings: MAPPO in particular benefits from high coordination via its CTDE paradigm. In the Harvest game, MAPPO and A2C prioritize greedy behaviour for immediate returns rather than delayed harvest which would maximize the total reward.

References

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[2] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[3] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 2094–2100, 2016.

[4] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, pages 1995–2003, 2016.

[5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[6] PyTorch Documentation. Reinforcement Q-Learning tutorial. `https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html`, 2024.